Fuzzing for Web Browser Under-Invalidation Bugs

Anonymous Submission

***Abstract*—Fuzzers are widely used to find security-related bugs in web browser parsers and media engines through crash testing, and to discover cross-browser and cross-version regressions via differential testing. We introduce a new type of browser correctness bug, under-invalidation, which occurs when a browser incorrectly updates its internal state after a style update. Under-invalidation bugs allow for *self-differential* testing, which tests a browser rendering engine against itself, leading to particularly clean fuzzing results.**

To find under-invalidation bugs efficiently, we develop LQC, a fuzzer that rapidly generates candidates web pages, tests them in a real browser via self-differential testing, and then minimizes and clusters any pages that witness under-invalidation bugs. In our evaluation, LQC is able to reproduce 7 human-submitted bugs mined from the Mozilla bug tracker with self-differential testing. LQC is also able to test hundreds of fuzzer candidates per minute in both Chrome and Firefox, finding dozens of bug candidates in each browsers (79 in Chrome and 125 in Firefox). Furthermore, LQC's bug minimization and clustering passes reduce the style-element volume by approximately 2.7x through minimization and 385x overall after clustering, all while requiring only 14.16% of the runtime. Finally, we show that LQC finds 15 novel bugs, across both browsers, that were reported to and accepted by the developers, with 7 already fixed.

***Index Terms*—Browser fuzzing, layout invalidation, under-invalidation, self-differential testing**

I. Introduction

Web browsers are central to modern computing, hosting applications that correspond to over **80%** of daily work in some organizations [1]. Browser makers therefore invest heavily in browser correctness, [2]–[7], including through fuzzing [8], which uses a large number of randomly generated inputs to identify bugs in the browser.

Browser developers use fuzzers to find both security and correctness bugs. On the security side, fuzzers such as Domato, Grammarinator, Fuzzowski, and MFFA [9]–[12] detect crashes in parsers, network libraries, and media engines, which are both exposed to untrusted input and must maximize performance. On the correctness side, fuzzers such as R2Z2 [13], Janus [14] and Metamong [15] use screenshots to detect browser rendering inconsistencies between different versions of the same browser, or between renderings of the different pages in the same browser, that may indicate a regression or rendering bug. However, all of these approaches struggle to precisely identify *under-invalidation bugs*, a subtle yet critical type of correctness bug in browser layout engines.

Invalidation refers to the process by which the browser updates internal data structures in response to style changes [16]. Since style changes are frequent (often every frame, 60 or even 120 times per second, in response to animations), substantial research has been devoted to designing efficient invalidation procedures [17], [18]. Generally, browsers attempt to update *as little* of their internal data structures as possible; however, updating *too little* of this internal state leads to an incorrect rendering that mixes fresh (updated) and stale (not updated) data. Updating exactly the right set of fields requires reasoning about transitive dependencies between layout algorithms, reasoning that is difficult enough that public bug trackers for Chrome, Firefox, and Safari's layout engines contain numerous under-invalidation bugs [19]–[24]. Modern invalidation algorithms like LayoutNG [25] are purposefully designed to reduce the risk of under-invalidation, while some promising algorithms [26] have failed precisely because under-invalidation bugs proved difficult to find and fix.

To combat these issues, this paper introduces a new oracle that detects under-invalidation bugs via *self-differential testing*, where either a large or small change results in the *same* web page, and where differences thus indicate under-invalidation, and validate this oracle on bugs mined from real-world browser bug databases. We extend this basic idea into LQC, a fuzzer that uses self-differential testing to detect under-invalidation bugs using random generation and minimization of complex web pages and web page changes, and leveraging automated bug clustering to reduce the bug review burden. We demonstrate that LQC is effective at both rapidly finding under-invalidation bugs (79 in Chrome and 125 in Firefox) and minimizing and clustering them, reducing the style-element review volume by approximately 385x. 15 bugs found by LQC were reported to Chrome and Firefox, all accepted by developers and 7 already fixed. LQC is now used as part of Mozilla's fuzzing workflow to find under-invalidation bugs during development.

This paper makes the following contributions:

- A new class of bugs, under-invalidation bugs, that can be efficiently detected using *self-differential* testing;
- LQC, a targeted fuzzer for under-invalidation bugs that generates web pages and web page changes, detects under-invalidation bugs with self-different testing, and minimizes and clusters the resulting bugs to reduce review load.
- An evaluation of LQC and its major components on both Chrome and Firefox, measuring bug discovery, minimization, and clustering with both direct and ablation studies.

We release LQC together with evaluation artifacts, test cases, and benchmarks at the following URL: https://anonymous.4open.science/r/lqc-7D637/.

II. Background, Related Work, and Motivation

This section introduces web browsers and layout, invalidation and under-invalidation bugs, and the challenges of fuzzing browser layout engines.

A. Web Browsers

Web browsers are among the most widely used and economically important software systems. A browser must load applications and content over the network; execute applications and react to user input; and render the application user interface to the screen. The rendering step is performed by the browser's *rendering engine* [16]; Fig. 1 is a high-level diagram of this rendering pipeline. The main production browser engines are Chrome's Blink engine [27], Firefox's Gecko engine [28], and Safari's WebKit engine [29], which power both the named browsers and many others, such as Microsoft Edge and Samsung Browser. Other browser-engine projects include Pale Moon, Ladybird, and Servo [30]–[32]. These engines require substantial and sustained investment; for example, Open Web Advocacy estimates that Google provides roughly 90% of Blink development funding and invests around $1 billion annually in the web platform [33].

This paper focuses on *layout*, which involves computing the size and position of every web page element. Layout is one of the most complex parts of the rendering pipeline [16], and, because browser rendering is performed after every user interaction, it must be highly optimized [17], [34], [35] Rendering engines achieve high performance via *invalidation*: most user interactions, animations, or APIs only affect a part of the web application's user interface, and the browser attempts to re-render only that portion of the page, re-using prior rendering results for the rest [16]. For example, suppose a user hovers over a menu, causing a dropdown to appear. The browser must recompute the position of the dropdown and all of the elements within it, but can reuse the size and position of the rest of the page, since the they are unaffected by the user action.

Invalidation leads to large speed-ups [17], but risks two kinds of bugs [34], [36]. *Under-invalidation* occurs when the browser fails to recompute changed portions of the user interface, causing a partially stale interface to be shown the user, which can confuse the user or even break application functionality. For example, the browser might recompute the dropdown's visibility status without recomputing its size and position, causing it to appear in the wrong location. Under-invalidation bugs often require a specific constellation of HTML, CSS, and JavaScript code to appear. For example, Chrome bug #40724799, found with LQC and since fixed by the Chrome developers, occurred required three nested elements, with the outermost having a `vertical-lr` writing style the middle one having a `inline-grid` display style, the innermost having a `horizontal-tb` writing style, and the application then setting the white-space style of the middle element to `break-spaces` and back. Chrome would fail to recompute an internal property called padding, requiring four rendering cycles to correctly compute the size and position of the middle element. Needless to say, constructing such an unlikely sequence by hand is difficult, making fuzzers valuable. Over-invalidation bugs, where the browser recomputes too much of the user interface, are also possible. That said, while these can slow rendering, they do not cause correctness issues and aren't a focus of this paper [25].

B. Browser Fuzzing

Fuzzers have a long history in browser testing, especially due to browsers' exposure to untrusted data over the network. JavaScript-engine fuzzers such as CodeAlchemist and DIE generate complex programs to expose crashes and security vulnerabilities [37], [38]. Grammar-based fuzzers such as Domato [9], by contrast, generate structured inputs to exercise deeper portions of the browser pipeline, while other browser fuzzers target specialized attack surfaces such as WebGL or WebAssembly [39], [40]. These approaches focus on detecting crashes, which often indicate a security vulnerability. For example, RTFuzz mutates render-tree state, similar to LQC, but primarily searches for crashes [41].

A newer collection of fuzzers instead target rendering correctness. For example, R2Z2, Metamong, Janus [13], [14] generate structured web pages and compare their differential rendering either across browsers or across browser versions. This differential testing approach can detect cross-browser compatibility issues, but are challenging to scale because different browsers or versions can differ for legitimate reasons, including different platform libraries, different user interface elements, or different supported features, among others. Under-invalidation bugs are especially ill-suited to a differential testing approach, because browser implement different invalidation algorithms. On the other hand, directly testing a browser layout against a known-good or reference implementation is not possible, because no browser is known to generate correct output in all cases and no reference implementation exists.

Under-invalidation bugs should be found by fuzzing. Existing browser fuzzers, which focus on differential testing, cannot easily find under-invalidation bugs. LQC uses self-differential testing instead.

III. Challenges and Solutions

To address the limitations of conventional browser fuzzing for layout under-invalidation bugs, we introduce LQC: a fuzzer that finds, minimizes, and groups under-invalidation bugs in product web browser layout engines. LQC's design is shaped by three key challenges.

A. Challenge 1: Defining a Correctness Oracle

The central challenge for layout correctness is in constructing an oracle that detects layout bugs.

a) No obvious failures: The web platform guarantees that web page layout is always a successful, atomic operation. There are neither failures nor intermediate states to observe. Crash-based and assertion-based fuzzers are thus insufficient to detect under-invalidation.

Fig. 1. A high-level diagram of the browser rendering pipeline. Of these steps, layout is most complex, as it instantiates the complex semantics of the CSS standard. The pipeline is rerun on every user interaction or animation, and invalidation is the main way browsers achieve high performance. Image reused with permission.

b) No ground truth: There is also no external ground truth to determine correct layout. Different browsers use different default fonts, different text layout libraries, and different user interface components. They also support different sets of web platform features, and use different implementation internals; for example, Firefox performs layout in sixtieths of a pixel while Chrome uses sixty-fourths of a pixel [**?**]. Plus, some aspects of layout, like table layout, are not fully specified. When two browsers disagree, this does not guarantee that either one is wrong. The Janus fuzzer also faced this issue [**?**], and compensated by using a weaker oracle: two different browsers should either both change or both not change their rendering in response to a user action. However, mistakes in invalidation often result in *partial* updates, making this weaker oracle insufficient to catch under-invalidation bugs.

c) Rapid change: Browser rendering engines also rapidly implement new features, making regression testing challenging. The R2Z2 fuzzer also faced this issue [**?**], and used the Web Platform Tests cross-browser test suite to differentiate between new features and new bugs. However, this is insufficient to detect new features that are implemented incorrectly, which is where detecting under-invalidation bugs is most valuable, an issue noted by the R2Z2 authors.

d) Self-differential testing: LQC addresses this challenge using self-differential testing oracle. The same HTML and CSS may be laid out correctly when the browser computes layout from scratch, but incorrectly when the browser reaches that state through an incremental update. But the web platform standards do not allow for path dependence; the layout of a web page should depend only on the page itself and browser parameters like viewport size. LQC thus compares the browser's layout of a page after incremental layout to the layout after from-scratch layout. Any difference indicates an under-invalidation bug. This approach compares browser rendering engine against itself, avoiding all sources of cross-browser and cross-version differences.

B. Challenge 2: Triggering Under-Invalidation Behavior

A fuzzer must not only detect when a bug has been triggered but also generate inputs that trigger them. Unfortunately, doing so for under-invalidation bugs is difficult, because these bugs often rely on complex situations to trigger.

a) Representing Changes: Invalidation algorithms consider both the web application state—the DOM, consisting of HTML elements and their CSS properties—and also the changes made to that state since the last layout. Fuzzing for under-invalidation bugs thus requires generating both. LQC thus generates a tree of HTML elements and, for each element, both a set of initial CSS properties and a set of changes to those CSS properties. Rendering the initial HTML and CSS and then performing the indicated changes can then trigger an under-invalidation bug.

b) Element relations: Under-invalidation bugs also depend on the relative position of elements in an application; a bug might involve an element have a certain CSS property and whose child has a certain other CSS property assigned. Children, grand-children, parents, siblings, and even more distant relationships affect layout and can thus be relevant to invalidation decisions. LQC thus generates large element trees where a multitude of elements with various relations are available.

c) Grammar Coverage: Under-invalidation bugs are most valuable to find during development, when algorithms are easiest to chance. Yet development typically focuses on new and then-little-used features. In the case of layout under-invalidation, this means support for the span of CSS properties is essential. For this reason, LQC is grammar-directed using Google Chrome's actual CSS properties grammar. The grammar-directed generation is used for both the initial CSS properties and for generating new CSS properties that are then assigned to change the application state.

d) Low Success Rate: Under-invalidation bugs are rare due to the high effort developers already go through to prevent them. This means most bug candidate do not trigger a bug, and makes fuzzing slow. Moreover, some under-invalidation bugs are known issues that are difficult to fix due to the structure of surrounding code; these bugs are less valuable to find than yet-unknown "long-tail" bugs, which are even rarer. To address this low success rate, LQC generates large web application states, with dozens of elements each with hundreds of initial CSS properties and hundreds of property changes.

e) Bug Complexity: While large application states are effective at raising the success rate for bug generation, the resulting application states are too complex for developers to effectively address the bugs found. LQC thus uses minimization to shrink successful bug candidates, removing elements, properties, and property changes, while preserving the candidate's bug-triggering behavior.

C. Challenge 3: Reducing Report Complexity

Some under-invalidation bugs can be traced to a simple logic error that developers can easily fix. Others can be useful for shaping the direction of work in progress, by identifying unworkable designs and driving the design process.

Others, however, are the consequence of developers previously selecting an unworkable design and now being unable to easily change it. Browser rendering engines are decades-old million-line code-bases, and all three types exist in abundance, but a fuzzer cannot easily distinguish between them.

a) Steering: Layout is complex and developers often work on one component at a time. Finding under-invalidation bugs in that specific component is then especially valuable, since they can be immediately fixed. This requires allowing the developer to influence the search space, such as by indicating CSS properties or CSS property values that the fuzzer should preferentially generate.

b) Summarization: Many fuzzer-generated bug candidates trigger the same underlying bug; tediously examining them all by hand, or even with AI assistance, is likely to be time-consuming. Since similar bug candidates—especially after minimization—likely trigger the same bug, clustering bugs by similarity reduces the review burden. Since under-invalidation bugs often depend on structural relationships in the element tree, the clustering technique should account for these relationships, along with the CSS properties or property values that are present within the cluster.

IV. Implementation

LQC is an end-to-end, feedback-free fuzzing pipeline for browser under-invalidation bugs. It detects under-invalidation using self-differential testing between invalidation and from-scratch layout; generates bug candidates using grammar-based generation of large element trees with many initial and changed properties; and minimizes and clusters bugs using both structural element relations and similar CSS properties. LQC uses Selenium WebDriver to drive a browser, and supports all three browser rendering engines: Chrome's Blink, Firefox's Gecko, and Safari's Webkit. Figure 2 summarizes the main stages of LQC pipeline.

A. Layout Oracle

For each bug candidate, LQC causes the browser, via Selenium, to render the initial application state (serialized as a web page). Importantly, the CSS properties for each element are serialized as HTML `style` attributes. This minimizes the amount of work performed by the matching and styling phases of the browser rendering pipeline (Figure 1), reducing the parts of the browser that could be implicated in any bug.

LQC then uses the Selenium APIs to perform the generated property changes and re-render the application state. The output of layout—the size and position of each element—is read using the browser's `getBoundingClientRect` API. Note that, unlike prior browser fuzzing tools [**?**], LQC does not use screenshots to compare renderings. The `getBoundingClientRect` API reads the output of layout without requiring them to be visually apparent, for example allowing it to detect when the size of a transparent element differs.

LQC then causes the browser to perform a from-scratch layout by executing the JavaScript command:

```
let root = document.documentElement;
root.innerHTML = root.innerHTML;
```

The `innerHTML` property used here is not a normal field access; instead, reads and writes to this property perform web application state serialization and deserialization, so this command has the effect of serializing the current (post-modification) state and then deserializing it, replacing the current state. This does not change the state (though see below) but it does cause every existing invalidation algorithm to invalidate the entire application state, recomputing the entire layout from scratch. LQC then records the new, from-scratch layout output using the same `getBoundingClientRect` API. Any difference between the first and second layout outputs is then treated as an under-invalidation bug.

In practice, a few extra tweaks were needed to make this bug oracle work. Firstly, the `content-visibility` attribute, which allows the browser to skip layout for off-screen elements.[1] This could produce false positives if the property changes affect which elements are onscreen. LQC thus never generates this (relatively rare) CSS property. Secondly, many CSS properties have multiple names; for example, `margin-left` and `margin-inline-start` are aliases when writing left-to-right. In Chrome, this triggered a bug, where serialization did not preserve the relative order of the two properties. Serialization and deserialization could thus change their order and thereby change the layout, without triggering under-invalidation. LQC thus avoids `margin-inline-start` and similar logical names.

To validate that this oracle captures real under-invalidation failures, we converted 11 historical under-invalidation bug report from Chrome and Firefox into equivalent LQC test cases, shown in Table I. We tested each historical bug report using the LQC oracle and found that it detected all 11 bugs, showing that the oracle is effective.

B. Test Case and Bug Generation

Internally, each LQC bug candidate is represented as a `RunSubject`:

- an `ElementTree`, which is a nested representation of the generated DOM;
- a base `StyleMap`, containing declarations applied to each element in the initial document; and
- a modified `StyleMap`, containing declarations applied later by JavaScript as a modification to the page.

Each component has a simple grammar-directed generator. The `ElementTree` is built from nested elements and text nodes, with each element generating a random number of children and each text nodes generating a random number of words. The generator probabilities are chosen to make the typical `ElementTree` dozens of elements. Generating text is important because the interaction of text and non-text

[1] It is intended for improving performance in long-scrolling "infinite scroll" user interfaces, where it allows the browser to skip layout for off-screen content.

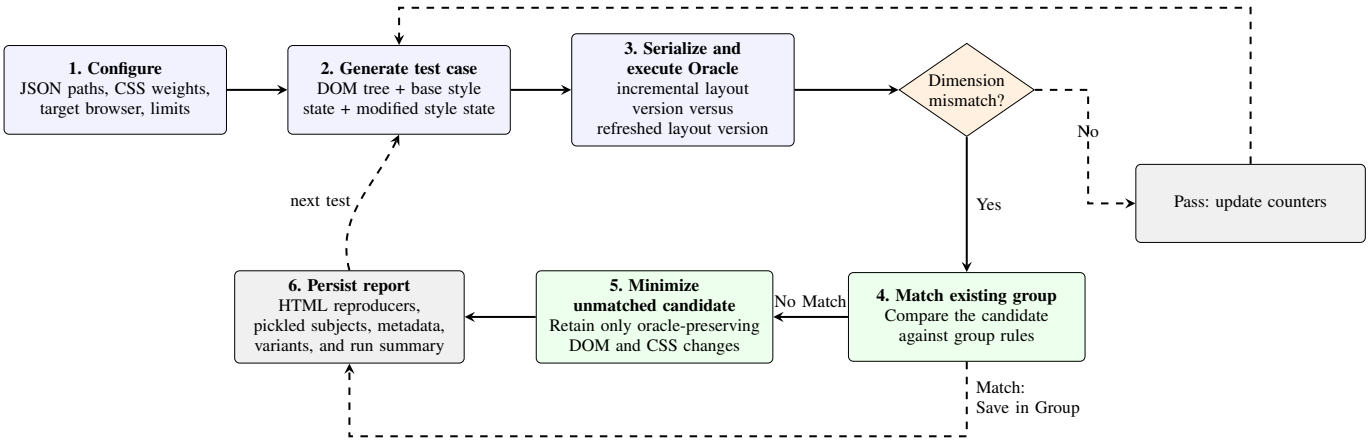

Fig. 2. LQC pipeline. Each test case is tested by a layout oracle. Passing subjects advance the fuzzing loop; candidates are structurally classified, minimized when necessary, and persisted with their reproduction artifacts and cumulative metrics.

content is a critical component of layout and a source of under-invalidation bugs. By contrast, different HTML element names are mostly unimportant, because an element's role in layout is driven by its CSS properties, not its tag name, so LQC sticks to generic `<div>` elements..[2]

The two `StyleMaps` are generated by selecting, for each CSS property, either to not generate a value for it or to generate a value based on that value's grammar as defined in Chrome's `css-properties.json5` parser generator. For keyword properties like `display`, LQC selects each valid keyword with some probability; for numeric properties like `width`, it selects a random value between -1000 and 1000 and adds a randomly-selected unit.[3] One exception to the above rule is that LQC always generates, for each element, a value for the `display` property. This property sets the "layout mode" of the element, which affects the entire layout computation. Selecting a diversity of layout modes across the page increases the amount of cross-layout-mode interactions, which is a common source of under-invalidation bugs.

A browser developer can affect the probability of including a property, and the probability of generating any particular grammatical construction, by setting probabilities in a configuration file. The Chrome developers made use of this capability when working on a redesign of the grid layout mode by biasing LQC toward generating complex grid layouts, and the Firefox developers did the same for the `flow-root` value of the `display` property when testing a reflow root invalidation change. Conversely, both browsers' developers chose to reduce the probability of `writing-mode` property, which chooses between horizontal and vertical writing, when vertical writing support was immature in both browsers, to reduce the rate at which known but low-priority bugs were discovered.

The generation step generates large bug candidates: dozens of elements with hundreds of properties each. The size is tuned so that bugs are found frequently, increasing the effectiveness of fuzzing; in effect, large bug candidates exercise multiple code paths and thus probe for many under-invalidation bugs at once. However, making bug candidates even bigger would be unwise: if a bug candidate contains two independent bugs, which one survives minimization would depend on the minimization algorithm, which could bias LQC away from certain (likely, more complex) bugs. LQC is tuned to detect a bug roughly once or twice a minute, which takes a few hundred bug candidates. As browser developers fix under-invalidation bugs and the rate of multiple bugs falls, LQC could be tuned to generate larger bug candidates and thus preserve its effectiveness.

C. Minimization and Structural Clustering

LQC's strategy of generating large bug candidates raises bug finding efficiency but makes the resulting bugs almost impossible for human developers to directly understand. It thus performs a minimization pass to simplify the bugs while preserving the detected under-invalidation bug (under the oracle). It does so by repeatedly performing simplifying transformations: removing elements; removing properties, either all at once or one at a time; and converting modified to base properties. Converting modified to base properties often results in a bug where only a single property is modified, which then helps enormously with debugging. Minimization also applies some readability-oriented transformations, like setting minimum sizes and adding background colors; like with all other minimization steps, the oracle is re-tested after every transformation.

As is common in fuzzing, many generated bugs exercise the same underlying root cause. In fact, many bugs are strikingly similar after minimization. To further reduce review burden, LQC thus attempts to cluster similar bugs. Each cluster is defined by a template; when a bug is found, it is matched against the templates of every existing cluster, and if one matches the bug is placed in that cluster without minimization. Developers can review the template instead of the individual bugs in the cluster. (Of course, they can also review individual

[2]The tag name plays a role in the earlier matching phase.

[3]Including both absolute units like `px` and relative units like `%`.

TABLE I
COMPONENTS OF CONVERTED BROWSER BUG TEST CASES.

Case	HTML tree	Initial styles	Style mutation
166194	`div > (text + div > div > text) + div`	`outer, inner, after: clear:both; action, text: float:left; middle: display:none; margin-bottom:10em`	`middle.display = inline-block`
648383	`div + div + div > div`	`first, resizeMe, third: float:left; resizeMe: width:100%`	`resizeMe.height = 100px`
1733276-2	`div > (text + div)`	`container: width:100px; test: display:none; margin-left:150px`	`test.display = inline`
1733276-3	`div > (text + div)`	`container: width:100px; test: display:none; padding-left:150px`	`test.display = inline`
Overflow/move	`div > div > div`	`absolute: position:absolute; left:200px; top:0; overflow:hidden; inner2: height:1000px; overflow:auto`	`left = 100px; top = 100px`
Letter spacing	`text + pre`	`pre: display:inline; letter-spacing:1em`	`width = 50px; layout; width = auto`
Table display	`table > tr > td > div`	`td: width:100px; height:100%; element: display:block; height:100%`	`element.display = table`
Scroll area	`div > (div + div)`	`scrollArea: 200px x 200px; overflow:scroll; abspos: position:absolute`	`scroll = (50,50); width = 220px`
Grid/table	`div > (div + (div > div) + div)`	`five: display:table-column; grid-auto-columns:20px; four: display:table; two: display:table-row`	`five.display = grid`
Multicol/flex	`div > (div + div > (div + div))`	`multicol: columns:4; height:100px; flexbox: display:flex; target: height:50px; contain:size`	`target.height = 900px`
Abspos inline	`text + span > div`	`span: position:relative; abspos: position:absolute`	`vertical-align = top`

Note: We took test cases from browser bug reports and converted them into equivalent LQC test cases. These conversions show that, as long as LQC can generate an HTML tree, its initial styles, and a style mutation, it can generate test cases that reproduce these bugs. Only declarations relevant to each mutation are shown.

bugs.) This allows them to rapidly skip known bugs and focus on the ones that are immediately fixable.

However, critically, the templates are discovered dynamically from the generated bugs instead of being defined by hand. Every time a new bug is found that does not belong to an existing cluster, LQC attempts to generalize every existing template to match the new bug. The generalized template is then checked against all *non-bug* bug candidates it has already generated;[4] if the new template does not match any non-bugs, the generalized template is kept, and the new bug is grouped with that cluster. On the other hand, if all generalized templates match non-bugs, the newly-generated bug assigned to a new cluster whose template exactly matches the new bug (and thus only matches it). In this way, the clusters always cover all bugs and no non-bugs, but one cluster may contain several related bugs, making clusters meaningful. In practice, most clusters achieve their final, most-general template after just two or three bugs, and generated templates are small and easy to review.

The technical representation of templates is critical for this to process. To form a template, two bugs' element trees are aligned at the first (and usually only) element with a modified property. These are always `<div>` elements, since text nodes cannot be assigned CSS properties. Then, the template includes those elements' parents, children, and siblings (and, recursively, their parents, children, and siblings) only to the extent that they exist and are the same type. So, for example, if the aligned elements in two bugs both have a text child followed by a `<div>` child, then the template will contain all three tree nodes, but if one element has a text child and the other has a ¡div¿ child, the template will not include either child, since their types differ. Then, for every remaining element, their base and modified CSS properties are kept only if both bugs assign a value to that property, and the value is kept only if both bugs assign the same one.[5]

In effect, in other words, templates are computed by "intersecting" two different bugs' element trees. They are likewise generalized by intersecting a new bug with the existing template. In our experience, this tree intersection algorithm, while somewhat unusual, is effective at finding short templates that apply to many bugs and do not apply to non-bug bug candidates, making them effective. In either case, note that the larger template generalization algorithm

[4]This is only done once at least 100 non-bug bug candidates exist, to ensure that this check is non-vacuous.

[5]For length properties, if both bugs assign the same kind of unit.

guarantees that clusters are meaningful; the tree intersection algorithm influences the number and size of clusters but not the correctness of the clustering algorithm itself.

The developer workflow for LQC is then as follows. The developer first, optionally, writes a configuration file describing the properties and property values that they want LQC to focus on (this raises the probability of generating such). They then launch LQC and run it for any length of time they please; as it runs, LQC records all bug candidates it generates, along with their status: non-bug bug candidates, used for testing generalized templates; and bugs, stored by cluster along with the cluster's template. The developer then reviews each cluster, likely by examining the cluster's template, and identifies the generated bugs they are able to fix.

V. Evaluation

Our evaluation of LQC answers three research questions:

RQ1 Is LQC effective at finding under-invalidation bugs?
RQ2 Does minimization and clustering reduce review load?
RQ3 Does LQC find novel under-invalidation bugs?

We test LQC on Google Chrome (149.0.7827.115) and Mozilla Firefox (152.0) using Selenium (4.40.0) via chromedriver (149.0.7827.115) and geckodriver (0.37.0). LQC itself runs on Python (3.13.11).

Because no existing fuzzer specifically targets under-invalidation bugs, our evaluation focuses on LQC 's efficiency and the quality of the results it produces. For RQ1, we fuzz each browser for 60 minutes, and additionally compare the impact of LQC's tweaks to generation of the `content-visibility` and `display` properties. For RQ2, we perform a longer, nine-hour fuzzing run on Mozilla Firefox and examine the size of original, minimized, and clustered bugs. For RQ3, we report on LQC-generated bugs that we have filed in the Google Chrome and Mozilla Firefox bug trackers. Note that RQ3 under-states the impact of LQC on Mozilla Firefox: after a pilot, the Mozilla developers chose to run LQC in Grizzly, Mozilla's internal fuzzing toolchain [**?**], so as to detect bugs before they make it into a user-visible release.

A. RQ1: Finding Under-Invalidation Reports

RQ1 asks whether LQC is effective at triggering under-invalidation behavior in Chrome and Firefox. As a secondary goal, we also test LQC's tweaks to the `content-visibility` and `display` properties, which are implemented as default property weights. Figure 3 shows bug counts over time for all four tested configurations. Every configuration finds dozens of reports within 60 minutes: Chromium finds 105 reports without property weights and 100 reports with property weights, while Firefox finds 63 and 137 reports, respectively. The large counts in each case demonstrate that LQC reliably finds under-invalidation bugs.

Table II shows numerical results for candidate generation speed, total bug count, and bug detection rate. Over all, LQC tests hundreds of bug candidates per second in both browsers and typically finds between one and two bugs per minute. This allows for a fine-grained comparison of the effect of LQC's default property weights. In Firefox, weighting raises the candidate count from 63 to 137 and the bug count from 0.212% to 0.463%, while throughput remains nearly unchanged (495.9 versus 492.8 tests per minute). In other words, the default weights allow LQC to find more bugs in Firefox at no additional cost. In Chromium, weighting increases throughput but does not improve discovery rate: the unweighted configuration produces more reports (105 versus 100) and a higher report-triggering rate (0.889% versus 0.299%). The difference is due to differences in how Chrome and Firefox implement the `content-visibility` property. Firefox's implementation does not generate false positives, so generating this property or not has little effect on Firefox; meanwhile, the higher probability of `display` raises the total number of bugs found. By contrast, Chrome's implementation of `content-visibility` does cause false positives, which causes LQC to fruitlessly attempt to minimize the false positive, until it is minimized all the way to zero modified properties, at which point the bug is useless. Disabling this property thus dramatically reduces the number of minimization steps needed, which explains the increased throughput, while also reducing false positives, explaining the reduced bug rate.

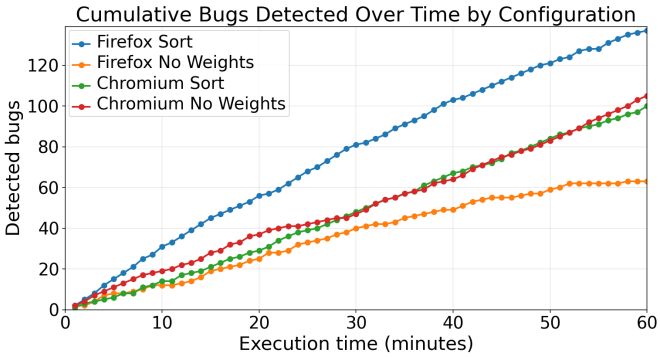

Fig. 3. Cumulative bug reports for the weighted and no-weight configurations over the 60-minute evaluation window.

TABLE II
Bug-discovery results across browser and configurations.

Browser	No weights			Weights		
	Speed	Bugs	Rate	Speed	Bugs	Rate
Chromium	196.8	105	0.889%	558.0	100	0.299%
Firefox	495.9	63	0.212%	492.8	137	0.463%

Note: Runs use a 60-minute execution window. Speed is measured in executed tests per minute; rate is bugs divided by executed tests.

RQ1: LQC finds under-invalidation reports in both Chromium and Firefox within one hour. Its default weights improve bug discovery, though their benefit is browser-dependent rather than universal.

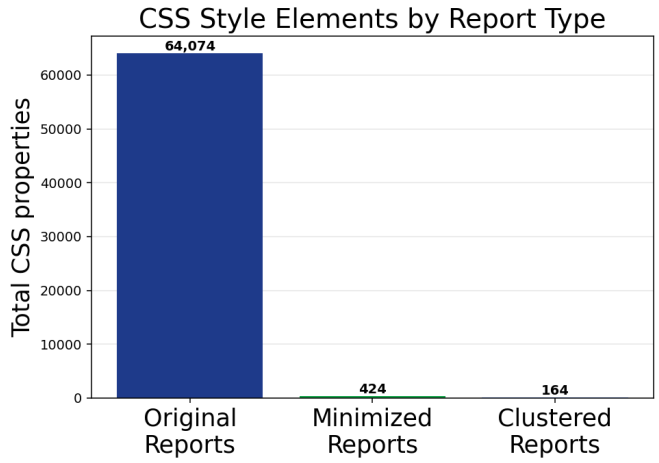

Fig. 4. Total CSS style elements in grouped Firefox reports before minimization, after minimization, and after clustering.

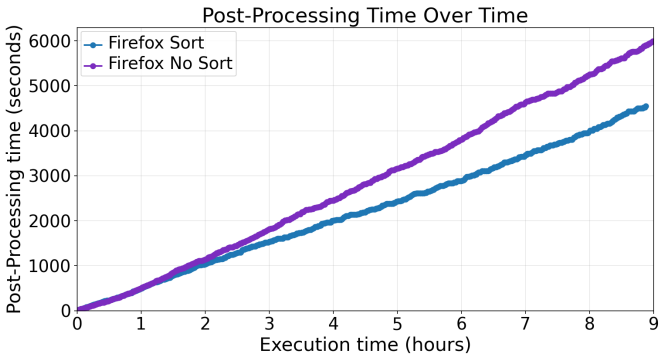

Fig. 5. Cumulative minimization and clustering time for Firefox sorting and no-sorting configurations.

B. RQ2: Reducing Developer Triage Work

A bug is most useful when it is easy to understand and analyze. RQ2 asks whether minimization and clustering reduce the burden of understanding and analyzing the bug. As a secondary goal, we asks about the impact of minimization and clustering on bug discovery speed. Both are measured on a nine-hour fuzzing run of Mozilla Firefox, with LQC's default weights. Figure 4 shows the size of the generated bugs, under three different conditions: without either minimization or clustering; after minimization but not clustering; and then after clustering, considering the template for each cluster. Size is measured by the total number of CSS properties in the base styles of the resulting bugs, though similar results would be seen for counting elements or modified styles. Minimization reduces the style-element volume in generated bugs by approximately 2.7x relative to the original reports. Combining minimization with clustering reduces the volume by a further 147x relative to minimized reports. Together, minimization and clustering produce an approximately 385x reduction in style-element volume, substantially lowering the review burden for developers using LQC.

The minimization-and-clustering pipeline accounts for 14.16% the nine-hour Firefox run's runtime. However, as shown in Figure 5, clustering actually reduces the minimization-and-clustering time: the clustering configuration executes ∼130,640 test cases, compared with ∼126,210 without clustering, and spends ∼4,550 seconds on minimization and clustering rather than ∼5,890 seconds without clustering. This is because, when a newly-generated bug matches the template of an existing cluster, LQC can skip minimizing the new bug. Consequently, clustering reduces cumulative post-processing time by ∼22.6% and enables ∼4,430 additional tests to run.

RQ2: Minimization and clustering substantially reduce the burden of reviewing generated bugs at relatively low additional elapsed time.

C. RQ3: Finding Confirmed Bugs

The bugs found in RQ1 and RQ2 are not, by themselves, evidence of distinct root-cause bugs. RQ3 therefore asks whether LQC finds a diverse collection of bugs that can survive external review.

Figure 6 shows the final distribution of bug groups for the nine-hour Firefox run in RQ2; there are 37 singleton bugs collected into a single bar. As is typical for fuzzers, the largest cluster is quite large, with 203 total bugs, roughly a third of all bugs found in this run, but the total number of bug clusters is large, showing that LQC finds a diversity of different bugs. Figure 7 shows how the number of bug groups and singleton bugs changes over time. After a rapid increase over the first three hours of the run, the number of singleton bugs reaches a rough steady state, as does the total number of bug groups. This is because, while LQC continues to test new bug candidates, and while a substantial fraction continue to trigger an under-invalidation bug, the clustering algorithm largely assigns these bugs to existing clusters. In a long enough run, we expect that LQC would generate multiple of every kind of bug that it could generate, and the number of singleton bugs would eventually approach 0 while the number of bug clusters would hit some maximum.[6]

Simply grouping bugs is not enough to confirm that the bugs are novel. Table III thus lists 15 bugs reported by the authors to the Google and Mozilla bug trackers, along with their bug tracker status as of the time of submission. All bugs have been accepted by the developers. 7 of these bugs have additionally been fixed by developers, showing that the bugs are not only valuable to developers but also fixable.

RQ3: LQC finds a substantial diversity of under-invalidation bugs, including 15 confirmed bugs, of which 7 have been fixed.

[6]Perhaps roughly 80, based on the number of bug groups plus the number of singleton bugs at the end of the run.

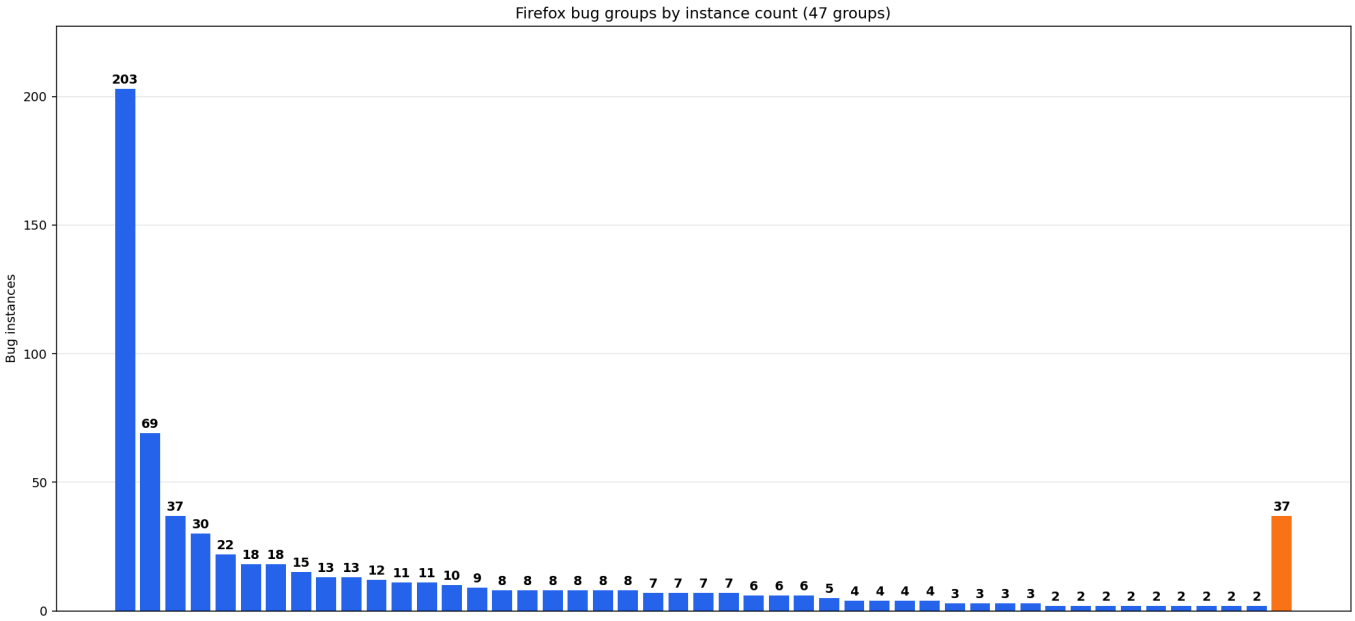

Fig. 6. Firefox bug-group sizes, with all single bugs combined into one bar.

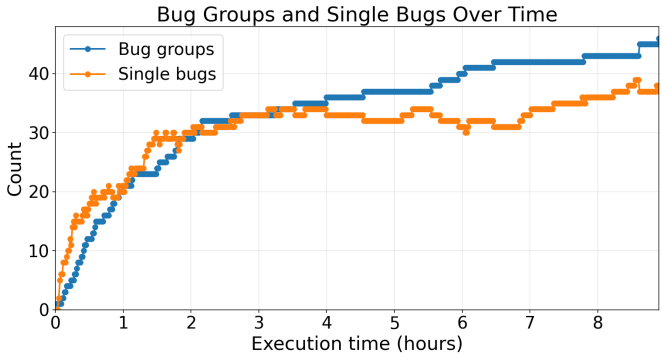

Fig. 7. Bug-group and single-bug counts by total detected reports in the Firefox sorting run.

TABLE III
STATUS OF BUG REPORTS SUBMITTED TO CHROME AND FIREFOX.

Browser	Bug ID	Brief description	Status
Chrome	1137427	Repeated white-space application failure	Fixed
Chrome	1166887	Dynamic table-caption vertical order swap	Fixed
Chrome	1189755	Fixed-position grid layout under-invalidation	Fixed
Chrome	1189762	None-to-inline-grid display invalidation	Open
Chrome	1190220	Grid layout page crash	Fixed
Chrome	1219641	Inline-grid list-item text-indent invalidation	Open
Chrome	1224721	Grid layout under-invalidation report	Fixed
Chrome	1229681	Grid child display-none invalidation	Open
Firefox	1721719	Table row-group height inconsistency	Open
Firefox	1724982	Table min-height overflow discrepancy	Open
Firefox	1724991	Dynamic break-spaces invalidation inconsistency	Open
Firefox	1733276	Display-inline margin line-wrapping inconsistency	Open
Firefox	1735376	Grid table-row height inconsistency	Fixed
Firefox	1735931	Percent-sized grid-item height inconsistency	Open
Firefox	1748891	Sticky padding scrollport position inconsistency	Fixed

VI. DISCUSSION & THREATS TO VALIDITY

A. Runtime Throughput

Test generation, minimization, and clustering all take time, reducing the overall throughput of the fuzzer. This is especially true of minimization, which requires many web page rendering steps to fully minimize a bug because it largely works step-by-step. A more efficient approach could use divide-and-conquer minimization. Instead of testing each element individually, the minimizer could first try removing larger groups of HTML nodes or CSS declarations, allowing large initial simplifications to be completed quickly. Throughput could also be improved by tuning the size of generated bug candidates, parallelizing and pipelining generation, rendering, and minimization, and distributing the fuzzing process across multiple machines. Naturally, all of those steps would add substantial complexity.

Alternatively, the LQC oracle and generation procedure could be added to existing browser fuzzing harnesses. This has already been done with Grizzly [**?**], the fuzzing hardness used by Mozilla; we would be interested in supporting other harnesses, such as Google's Clusterfuzz [8]. In any case, LQC is currently effective enough to find under-invalidation bugs in real-world web browsers in an overnight run on a single machine, making this investment of engineering effort currently unnecessary.

B. Browser Coverage

LQC's results depend on which browsers are supported and tested. Mainstream browsers like Chrome, Firefox, and Safari are supported via Selenium WebDriver, but newer browser rendering engines like Ladybird, Servo, or Flow may not be supported. That said, these more experimental rendering

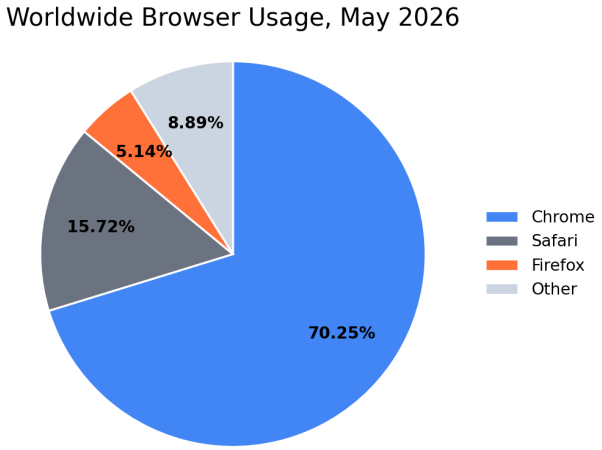

Fig. 8. Worldwide browser usage in May 2026 [43].

engines also pursue less aggressive invalidation strategies, meaning under-invalidation testing may more less important. As these browsers mature, we expect them to gain support for the WebDriver standard and thus become testable via LQC. Similarly, testing older versions of mainstream browsers may be difficult due to gaps in the WebDriver compatibility matrix. Thus, LQC may not be usable for analyzing changes in under-invalidation bugs over time. But, all told, recent versions of Chrome, Safari, and Firefox account for over 90% of web browser usage (**??**), and many less-common browsers share rendering engines with one of the mainstream browsers; experimental browsers in particular are extremely rare in practice.

C. Novel Bug Discovery Rate

The novel bugs discussed in Section V cover only a fraction of total bugs found by LQC, since our collaborators at Mozilla preferred to transfer LQC to using their internal fuzzing workflow. This makes it difficult to know exactly how many bugs LQC has found, whether those bugs are novel, and whether developers find it useful in their day-to-day work. In particular, it is difficult to know whether the clustering algorithm can be modified to further group related bugs and thus further reduce the review burden for developers. It's also possible that the templates from bug clusters could be used to modify the probabilities with which bug candidates are generated, so as to reduce the rate at which duplicate bugs are found and increase LQC's exploration of novel parts of the bug candidate search space. Despite these possible future improvements, the current random generation strategy remains useful because it discovers unexpected bug classes without requiring prior knowledge of effective configurations.

VII. Conclusion

Acknowledgments

References

[1] F. Montenegro, "The state of workforce security: Key insights for it and security leaders," Palo Alto Networks, Tech. Rep., Jan. 2025.

[2] Google Project Zero, "Google project zero," Google Project Zero, accessed June 27, 2026. [Online]. Available: https://projectzero.google/

[3] B. Grinstead, C. Holler, and F. Braun, "Behind the scenes: Hardening Firefox with Claude Mythos Preview," Mozilla Hacks, May 2026, published May 7, 2026. [Online]. Available: https://hacks.mozilla.org/2026/05/behind-the-scenes-hardening-firefox/

[4] World Wide Web Consortium, "Web standards," W3C, accessed June 27, 2026. [Online]. Available: https://www.w3.org/standards/

[5] Web Hypertext Application Technology Working Group, "Standards," WHATWG, accessed June 27, 2026. [Online]. Available: https://spec.whatwg.org/

[6] web-platform-tests project, "web-platform-tests documentation," web-platform-tests, accessed June 27, 2026. [Online]. Available: https://web-platform-tests.org/

[7] R. Andrew, "Interop 2026: Continuing to improve the web for developers," web.dev, Feb. 2026, published February 12, 2026. [Online]. Available: https://web.dev/blog/interop-2026

[8] Google, "ClusterFuzz: Scalable fuzzing infrastructure," GitHub, accessed June 27, 2026. [Online]. Available: https://github.com/google/clusterfuzz

[9] I. Fratric, "Domato: A DOM fuzzer," GitHub, 2017, accessed June 27, 2026. [Online]. Available: https://github.com/googleprojectzero/domato

[10] R. Hodován, Á. Kiss, and T. Gyimóthy, "Grammarinator: An ANTLR v4 grammar-based test generator," GitHub, 2018, accessed June 27, 2026. [Online]. Available: https://github.com/renatahodovan/grammarinator

[11] M. Rivas, "Fuzzowski: Network protocol fuzzer," GitHub, 2019, accessed June 27, 2026. [Online]. Available: https://github.com/nccgroup/fuzzowski

[12] MFFA contributors, "MFFA: Media fuzzing framework for Android," GitHub, 2015, accessed June 27, 2026. [Online]. Available: https://github.com/fuzzing/MFFA

[13] S. Song, J. Hur, S. Kim, P. Rogers, and B. Lee, "R2Z2: Detecting rendering regressions in web browsers through differential fuzz testing," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. ACM, 2022.

[14] C. Zhou, Q. Zhang, B. Qian, and Y. Jiang, "JANUS: Detecting rendering bugs in web browsers via visual delta consistency," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, ser. ICSE '25. IEEE, 2025.

[15] S. Song and B. Lee, "Metamong: Detecting render-update bugs in web browsers through fuzzing," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '23. ACM, 2023.

[16] P. Panchekha and C. Harrelson, *Reusing Previous Computations*. Oxford University Press, 2024, pp. 443–494. [Online]. Available: https://browser.engineering/invalidation.html

[17] M. Kirisame, T. Wang, and P. Panchekha, "Spineless traversal for layout invalidation," *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 1791–1813, 2025.

[18] J. Liu, Y. Chen, E. Atkinson, Y. Feng, and R. Bodik, "Conflict-driven synthesis for layout engines," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 132:1–132:22, 2023.

[19] Chromium, "Issue 497183437," Chromium Issue Tracker. [Online]. Available: https://issues.chromium.org/issues/497183437

[20] ——, "Issue 379886422 resources," Chromium Issue Tracker. [Online]. Available: https://issues.chromium.org/issues/379886422/resources

[21] Mozilla, "Bug 1452426," Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=1452426

[22] ——, "Bug 539356," Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=539356

[23] WebKit, "Bug 233421," WebKit Bugzilla. [Online]. Available: https://bugs.webkit.org/show_bug.cgi?id=233421

[24] ——, "Bug 97801," WebKit Bugzilla. [Online]. Available: https://www2.webkit.org/show_bug.cgi?id=97801

[25] I. Kilpatrick and K. Ishii, "RenderingNG deep-dive: LayoutNG," Chrome for Developers, 2021, last updated October 8, 2021. [Online]. Available: https://developer.chrome.com/docs/chromium/layoutng?hl=en

[26] MozillaWiki, "Gecko: Reflow refactoring," MozillaWiki, 2006, landed December 7, 2006. [Online]. Available: https://wiki.mozilla.org/Gecko:Reflow_Refactoring

[27] The Chromium Projects, "Blink: Rendering engine," The Chromium Projects, accessed June 2, 2026. [Online]. Available: https://www.chromium.org/blink/

[28] Mozilla, "Gecko," Firefox Source Docs, accessed June 2, 2026. [Online]. Available: https://firefox-source-docs.mozilla.org/overview/gecko.html
[29] WebKit, "Webkit," WebKit, accessed June 2, 2026. [Online]. Available: https://webkit.org/
[30] Pale Moon Project, "Pale moon: Custom, private, open-source web browsing," Pale Moon, accessed June 27, 2026. [Online]. Available: https://www.palemoon.org/
[31] Ladybird Browser Initiative, "Ladybird," Ladybird, accessed June 27, 2026. [Online]. Available: https://ladybird.org/
[32] Servo Project Developers, "About servo," Servo, accessed June 27, 2026. [Online]. Available: https://servo.org/about/
[33] Open Web Advocacy, "Break google's search monopoly without breaking the web," Open Web Advocacy, Apr. 2025, published April 23, 2025. [Online]. Available: https://open-web-advocacy.org/blog/break-googles-search-monopoly-without-breaking-the-web/
[34] M. Kosaka, "Inside look at modern web browser, part 3," Chrome for Developers, 2018. [Online]. Available: https://developer.chrome.com/blog/inside-browser-part3
[35] MDN Web Docs, "Web performance," MDN Web Docs, accessed June 2, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Performance
[36] Mozilla, "Performance best practices for firefox front-end engineers," Firefox Source Docs, accessed June 2, 2026. [Online]. Available: https://firefox-source-docs.mozilla.org/performance/bestpractices.html
[37] H. Han, D. Oh, and S. K. Cha, "CodeAlchemist: Semantics-aware code generation to find vulnerabilities in JavaScript engines," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS '19, 2019.
[38] S. Park, W. Xu, I. Yun, D. Jang, and T. Kim, "Fuzzing JavaScript engines with aspect-preserving mutation," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2020.
[39] H. Peng, Z. Yao, A. Amiri Sani, D. J. Tian, and M. Payer, "GLeeFuzz: Fuzzing WebGL through error message guided mutation," in *Proceedings of the USENIX Security Symposium*, 2023.
[40] O. Draissi, T. Cloosters, D. Klein, M. Rodler, M. Musch, M. Johns, and L. Davi, "Wemby's web: Hunting for memory corruption in WebAssembly," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, 2025.
[41] Y. Zeng, Y. Wu, X. Lu, and C. Zhang, "RTFuzz: Fuzzing browsers via efficient render tree mutation," *Computers & Security*, vol. 161, p. 104756, 2026.
[42] S. Kim, Y. M. Kim, J. Hur, S. Song, G. Lee, and B. Lee, "FuzzOrigin: Detecting UXSS vulnerabilities in browsers through origin fuzzing," in *Proceedings of the 31st USENIX Security Symposium*, ser. USENIX Security '22. USENIX Association, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/kim
[43] StatCounter GlobalStats, "Browser market share worldwide," StatCounter GlobalStats, May 2026, worldwide browser market share, May 2026; accessed June 24, 2026. [Online]. Available: https://gs.statcounter.com/browser-market-share